

Implementação do sistema de arquivos

Universidade Estadual de Mato Grosso do Sul

Bacharelado em Ciência da Computação

Sistemas Operacionais

Prof. Fabrício Sérgio de Paula

Tópicos

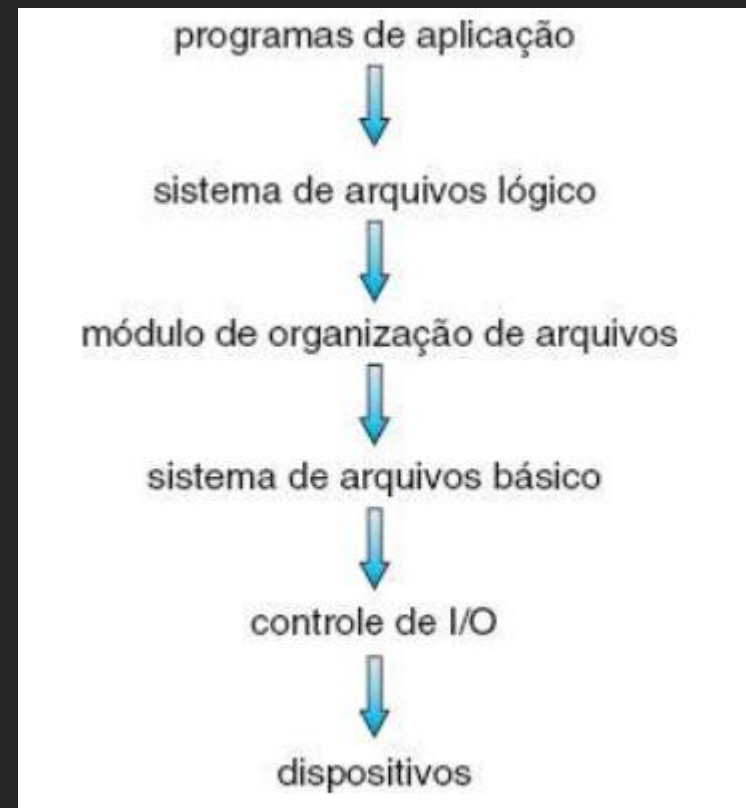
- Estrutura do sistema de arquivos
- Implementação do sistema de arquivos
- Implementação de diretórios
- Métodos de alocação
- Gerenciamento de espaço livre
- Eficiência e desempenho
- Recuperação

Estrutura do sistema de arquivos

- Sistema de arquivos: acesso eficiente e conveniente aos dados armazenados
 - Dois problemas de projeto a resolver:
 - Definição da aparência do sistema de arquivos ao usuário: definição de arquivo, atributos, operações, estrutura de diretórios
 - Criação de algoritmos e estruturas de dados para mapear o sistema de arquivos lógico para os dispositivos físicos
 - Sistemas de arquivos são organizados em diversos níveis/camadas: níveis mais baixos criam recursos para os mais altos

Estrutura do sistema de arquivos

- Sistemas de arquivos em camadas:



Estrutura do sistema de arquivos

- Sistemas de arquivos em camadas:
 - Controle de E/S: drivers de dispositivos e manipuladores de interrupção
 - Usa instruções de baixo nível para lidar com hardware
 - Sistema de arquivos básico: comandos genéricos para ler e gravar blocos físicos em disco e gerenciar buffers/caches
 - Adiciona eficiência ao sistema de arquivos
 - Módulo de organização de arquivos: conhece arquivos, blocos lógicos e físicos
 - Traduz endereços de blocos lógicos para físicos e vice-versa:
 - Bloco lógico: numerados de 0 a N
 - Blocos físicos: drive, cilindro/trilha, setor
 - Gerencia blocos livres e alocados

Estrutura do sistema de arquivos

- Sistemas de arquivos em camadas (cont.):
 - Sistema de arquivos lógico: gerencia estrutura de diretórios e metadados
 - Associa nomes aos arquivos
 - Metadados: toda estrutura do sistema de arquivos, exceto dados reais
 - Define bloco de controle de arquivo (FCB ou inode no Unix):
 - Informações sobre o arquivo: proprietário, permissões, localização dos dados

Estrutura do sistema de arquivos

- Vantagem da abordagem em camadas: evita duplicar código
 - Ex.: mesmo código de controle de E/S e até sistema de arquivos básico pode ser utilizado por sistemas de arquivos lógicos distintos
- Desvantagem da abordagem em camadas: overhead adicional

Implementação do sistema de arquivos

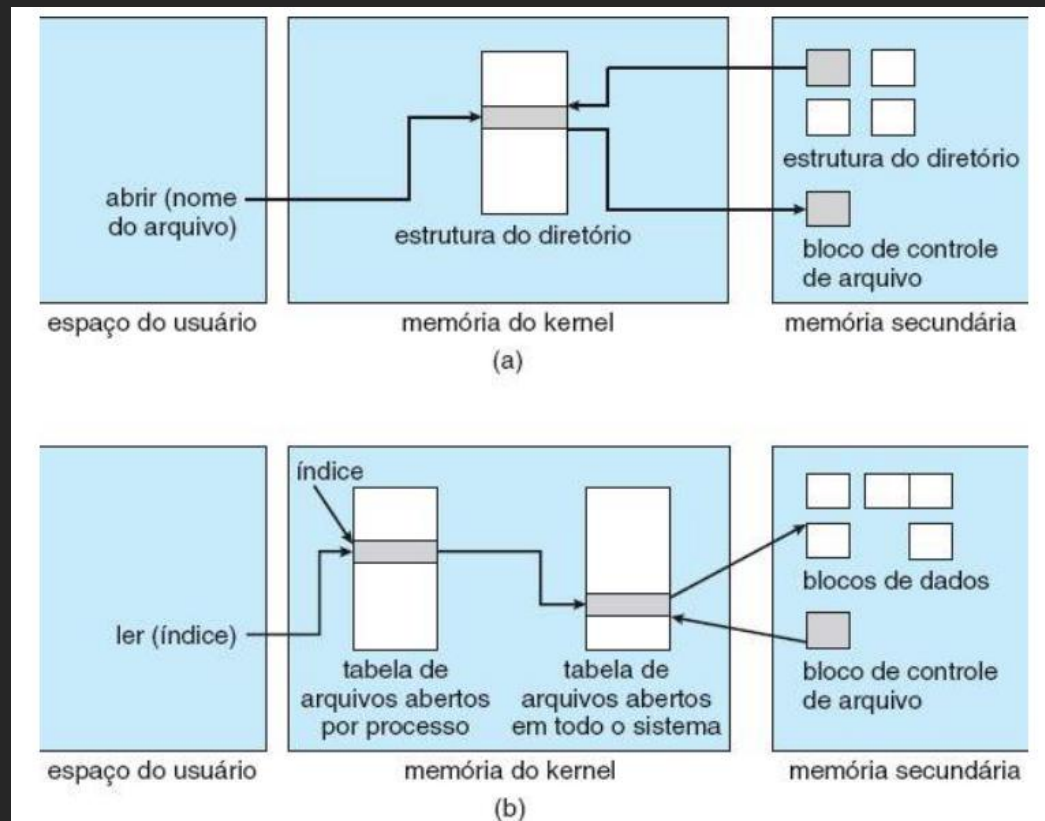
- Implementação de sistemas de arquivos faz uso de várias estruturas de dados em disco e memória
 - Em disco:
 - Bloco de controle de inicialização: contém informações para inicializar SO partir do volume/partição
 - Usualmente primeiro bloco do volume
 - Vazio quando não possui SO instalado
 - Bloco de controle de volume: contém número de blocos na partição, tamanho dos blocos, gerenciamento de blocos livres/alocados
 - Estrutura de diretório (por sistema de arquivos): organiza arquivos e adiciona nome aos arquivos
 - Bloco de controle de arquivo (FCB/inode): possui informações detalhadas sobre arquivo e identificador para associá-lo a entrada de diretório

Implementação do sistema de arquivos

- (cont.):
 - Em memória:
 - Tabela de montagens: informações sobre cada volume montado
 - Tabela de arquivos abertos no sistema: possui cópia do FCB de cada arquivo aberto
 - Tabela de arquivos abertos por processo: referencia entradas da tabela de arquivos abertos no sistema
 - Buffers: blocos do sistema de arquivos quando estão sendo lidos/gravados

Implementação do sistema de arquivos

- Estruturas de dados na abertura (a) e leitura (b) de arquivo



Implementação do sistema de arquivos

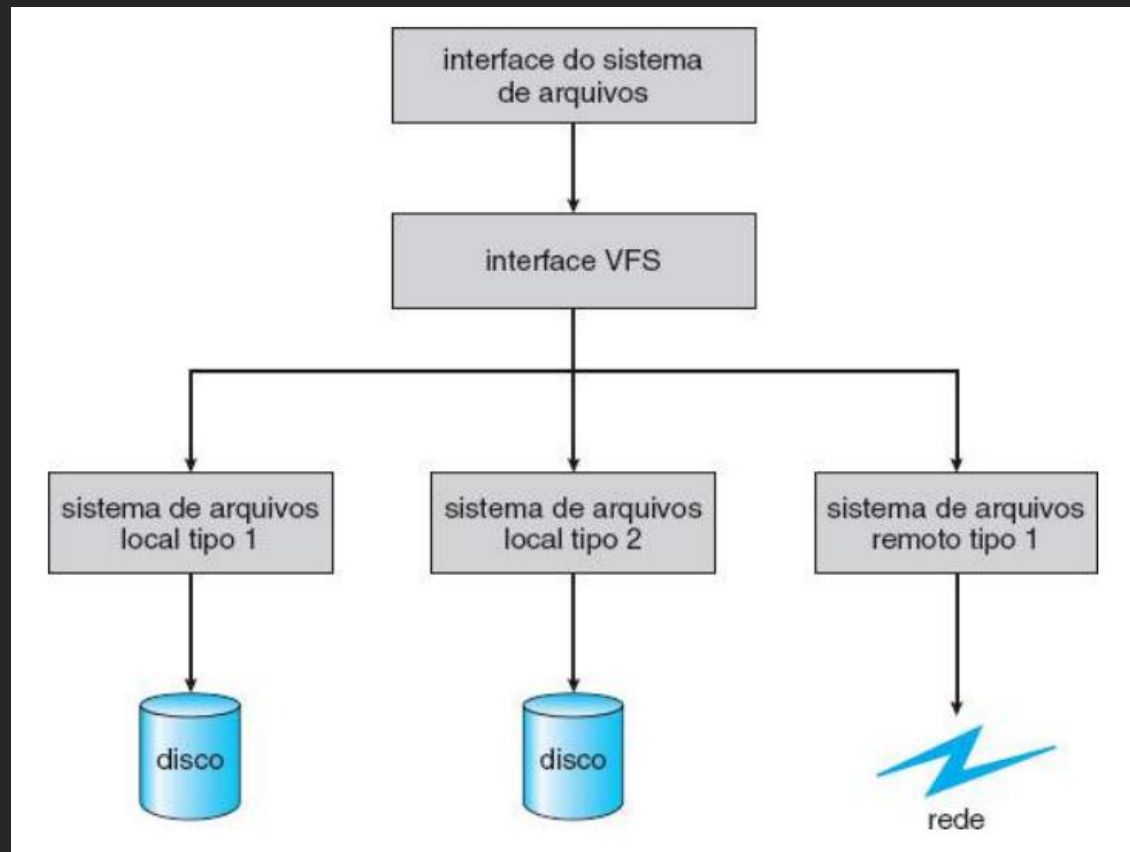
- Partições e montagem:
 - Partições podem ser “brutas” ou “acabadas”
 - “Brutas”: sem sistemas de arquivos
 - Ex.: Partição de swap, partição para BD (sistema de arquivos criado/gerenciado pelo próprio SGBD)
 - Durante boot:
 1. Carregador de inicialização (armazenado em local pré-definido do disco) é carregado na memória e inicia execução
 2. Carregador de inicialização, que suporta sistema de arquivos, localiza e carrega o kernel: pode suportar dual-boot
 3. Kernel inicia execução
 - Partição raiz (contém o kernel) é montada em tempo de inicialização
 - Demais partições são montadas posteriormente

Implementação do sistema de arquivos

- Sistema de arquivo virtual: permite acessar sistemas de arquivos distintos com mesma interface
- Implementação do sistema de arquivos é dividida em 3 camadas:
 1. Interface do sistema de arquivos: composta pelas chamadas `open()`, `read()`, `write()`, `close()`
 2. Sistema de arquivos virtual (VFS): apresenta interface genérica para acessar sistemas de arquivos diferentes
 - Separa operações genéricas de sistemas de arquivos da implementação propriamente dita
 - Várias implementações da interface VFS coexistem para suportar diferentes sistemas de arquivos
 - Identifica arquivo em rede: `vnode`
 3. Sistemas de arquivos específicos

Implementação do sistema de arquivos

- Sistema de arquivo virtual:



Implementação do sistema de arquivos

- Sistema de arquivo virtual:
 - Implementação usa técnicas de orientação a objetos
 - No Linux:
 - VFS define 4 tipos de objetos
 - inode: representa arquivo
 - arquivo: representa arquivo aberto
 - superbloco: representa sistema de arquivos inteiro
 - dentry: entrada de diretório
 - Para cada objeto é definido um conjunto de operações que devem ser implementadas
 - Para objeto arquivo: `open()`, `read()`, `write()`, `close()`, `mmap()`, etc.
 - Devem ser implementadas para sistemas de arquivos específicos
 - Operações são invocadas através de tabela de ponteiros para funções: permite executar operações específicas de forma genérica

Implementação de diretórios

- Implementação de diretórios: algoritmos e estruturas de dados para alocação/gerenciamento impactam no desempenho e confiabilidade
 - Lista linear: mais simples, mas ineficiente
 - Tabela hash: adiciona mais agilidade

Implementação de diretórios

- Implementação através de lista linear:
 - Usa lista linear de nomes de arquivos com ponteiros para blocos de dados
 - Criação de novo arquivo:
 - Pesquisa pelo nome do arquivo no diretório: deve ser único
 - Adiciona uma nova entrada no final da lista
 - Excluir de arquivo:
 - Pesquisa pelo nome do arquivo no diretório
 - Várias opções para remover entrada:
 - Marcar como “livre”
 - Anexar em lista de entradas livres
 - Copiar última entrada da lista para entrada removida: reduz tamanho do diretório

Implementação de diretórios

- Implementação através de lista linear (cont.):
 - Problema: busca por um arquivo exige busca linear
 - Operações de busca são muito frequentes!
 - Para minimizar problema:
 - Caches e memória de diretórios mais recentemente usados
 - Manter lista ordenada: dificulta criação e exclusão de arquivos
 - Usar árvore balanceada...

Implementação de diretórios

- Implementação usando tabela hash:
 - Lista linear armazena entradas de diretórios
 - Tabela hash é usada para acelerar busca
 - Nome do arquivo é a chave de busca
 - Entrada na tabela aponta para nó da lista linear
 - Colisões podem ser resolvidas com encadeamento externo (lista linear): aumenta o tempo de busca, mas continua sendo vantajoso usar tabela

Métodos de alocação

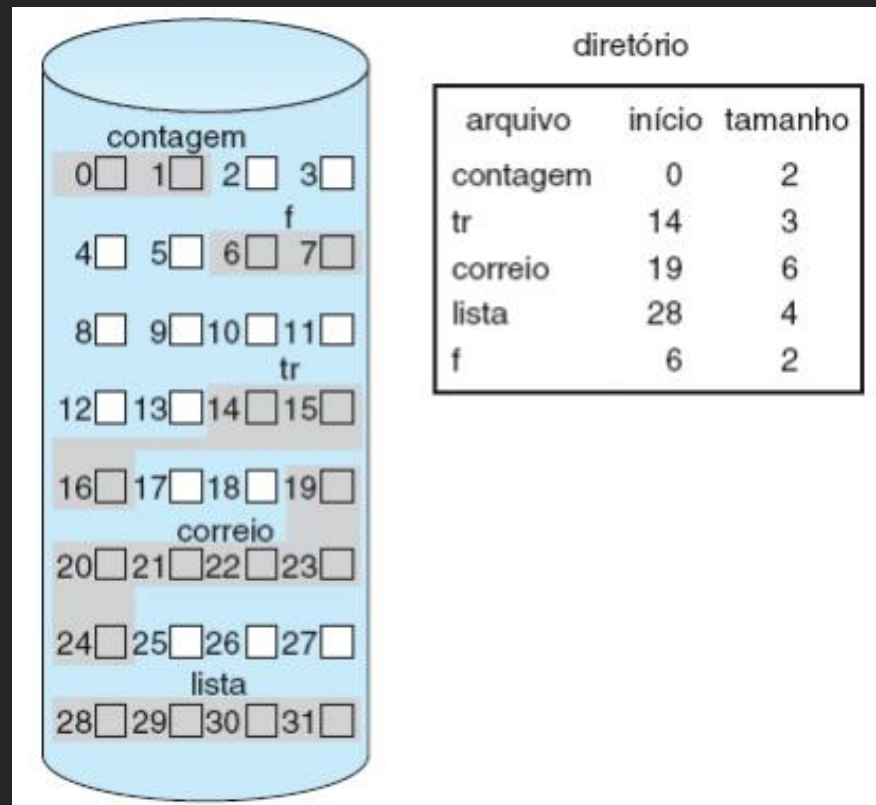
- Alocação de espaço em disco deve:
 - Utilizar espaço de forma eficiente
 - Permitir acesso rápido aos arquivos
- Métodos de alocação:
 - Alocação contígua
 - Alocação encadeada
 - Alocação indexada

Métodos de alocação

- Alocação contígua: arquivo deve ocupar blocos contíguos no disco
 - Usualmente não requer movimentação do cabeçote: após último setor, apenas para próxima trilha
 - Número de buscas no disco é mínimo
- Entrada de diretório deve conter:
 - Endereço do bloco inicial
 - Quantidade de blocos alocados

Métodos de alocação

- Alocação contígua (cont.):



Métodos de alocação

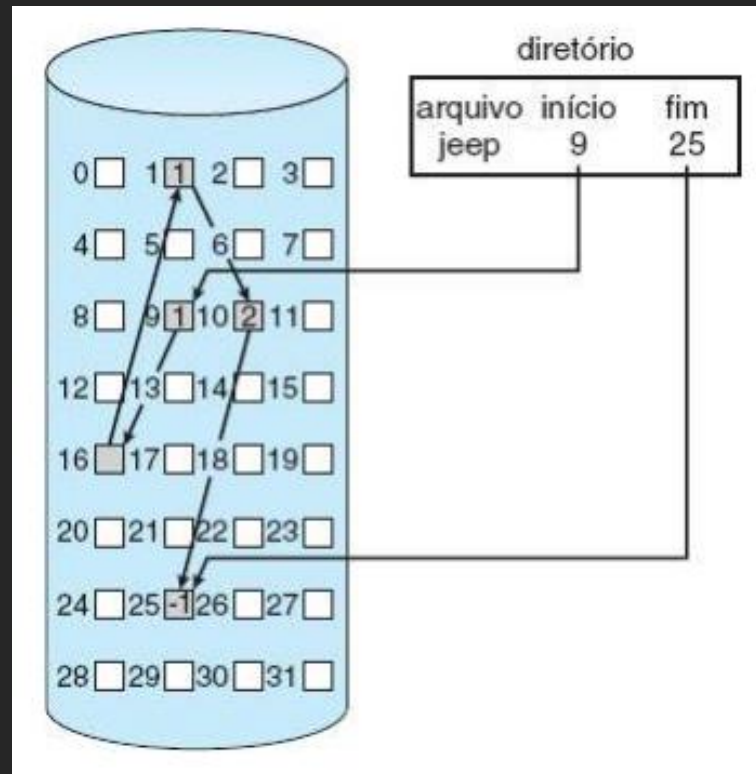
- Alocação contígua (cont.):
 - Encontrar espaço para novo arquivo: problema similar ao da alocação de memória dinâmica
 - First-fit, best-fit, worst-fit
 - Todos sofrem de fragmentação externa!
 - Solução: desfragmentação
 - Outro problema: tamanho do arquivo pode aumentar
 - Difícil estimar
 - Com best-fit fica pior: sobra menos espaço
 - Alguns SOs trabalham extensão: caso não seja possível aumentar, é criada extensão
 - Extensão: outra porção contígua separada da alocação inicial
 - Localização do arquivo: deve conter também endereço inicial e tamanho da extensão

Métodos de alocação

- Alocação encadeada: arquivo é uma lista encadeada de blocos de disco
 - Resolve problemas da alocação contígua
 - Entrada de diretório contém:
 - Ponteiros para primeiro e último blocos do arquivo
 - Cada bloco do arquivo contém ponteiro para o próximo bloco
 - Ponteiros não são acessíveis ao usuário

Métodos de alocação

- Alocação encadeada (cont.):



Métodos de alocação

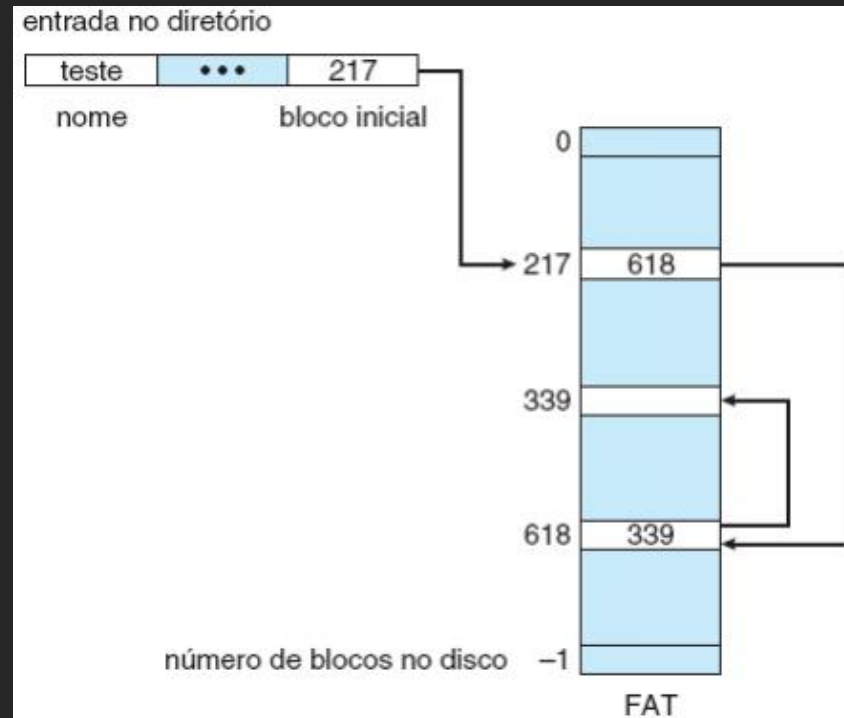
- Alocação encadeada (cont.):
 - Problemas:
 - Acesso ao i-ésimo bloco requer percorrer lista no disco
 - Acesso direto é ineficiente
 - Espaço gasto para armazenar ponteiros
 - 4 bytes de ponteiro em bloco de 512 bytes = 0,78%
 - Confiabilidade: alteração de ponteiro (falha de software ou hardware) pode levar ao acesso a blocos arbitrários (alocados ou não)
 - Pode ser melhorada usando lista duplamente encadeada: mais overhead
 - Alternativa para minimizar problemas: alocação de clusters
 - Cluster: alocar múltiplos blocos contíguos (ex.: 4 blocos por cluster)
 - Menos espaço gasto por ponteiros: um/dois por cluster
 - Acesso mais rápido: menos movimentações do cabeçote
 - Problema: maior fragmentação interna (último cluster)

Métodos de alocação

- Alocação encadeada:
 - FAT (file allocation table) do MS-DOS: variação da alocação encadeada
 - Tabela é armazenada no início de cada volume
 - Uma entrada para cada bloco em disco: indexada pelo número do bloco
 - Cada entrada armazena o número do próximo bloco do arquivo
 - Bloco não usado: valor 0
 - Entrada de diretório: número do bloco inicial
 - Desvantagem: se a tabela não estiver em cache, 2 acessos a disco para acessar bloco
 - 1 para acessar tabela + 1 para acessar bloco
 - Vantagem: tempo de acesso randômico é melhorado
 - Locação dos blocos pode ser realizada lendo apenas a tabela, ao invés de percorrer lista

Métodos de alocação

- Alocação encadeada (cont.):
 - FAT:

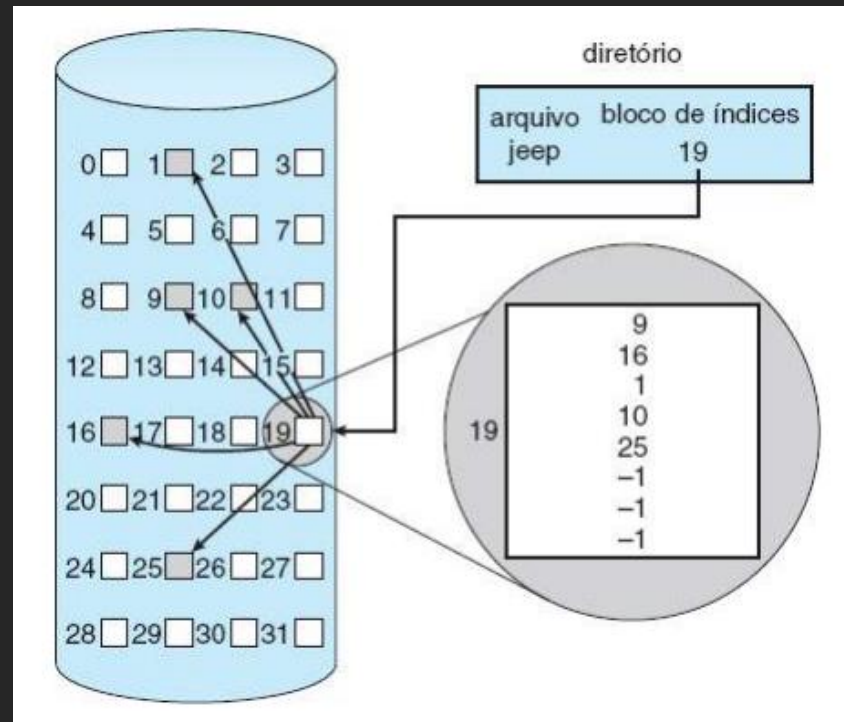


Métodos de alocação

- Alocação indexada: cada arquivo possui bloco de índices
 - Possibilita acesso direto sem sofrer de fragmentação externa
 - Bloco de índices: array contendo o número de cada bloco do arquivo
 - i-ésima entrada aponta para o i-ésimo bloco do arquivo
 - Quando arquivo é criado: apontadores são NULL
 - Entrada do diretório: aponta para bloco de índices

Métodos de alocação

- Alocação indexada (cont.):

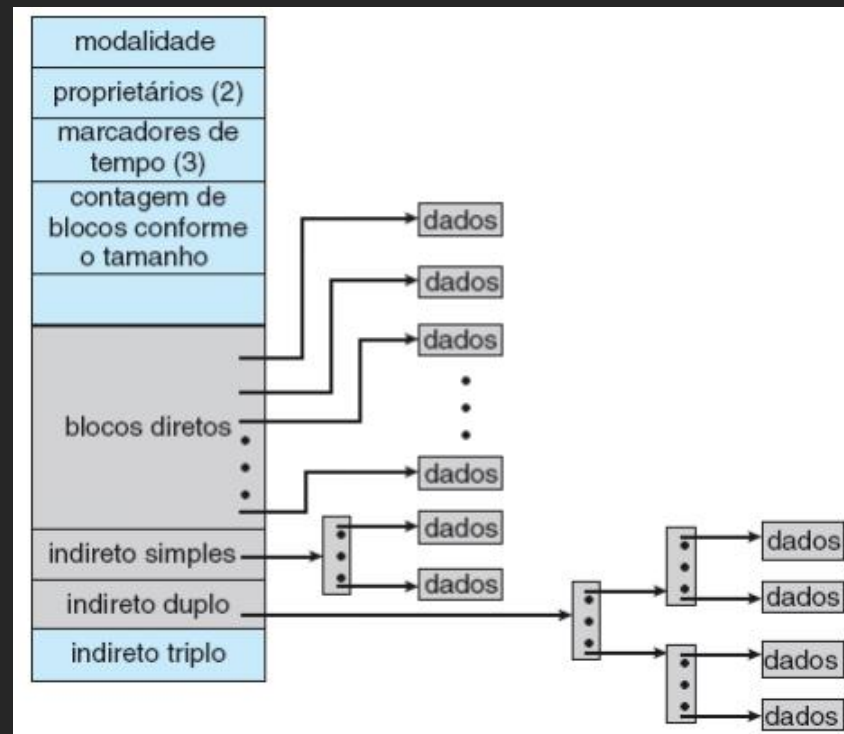


Métodos de alocação

- Alocação indexada (cont.):
 - Qual tamanho do bloco de índices?
 - Arquivo pequeno: muito desperdício de espaço
 - Arquivo grande: bloco de índices pode não comportar todos os ponteiros para blocos do arquivo
 - Soluções:
 - Esquema encadeado: encadeamento de blocos de índice de um mesmo arquivo
 - Índices multiníveis: bloco de índices do primeiro nível aponta para blocos de índices do segundo nível, que apontam para blocos do arquivo
 - Esquema combinado: combina soluções anteriores
 - Sistemas baseado no UNIX: inode guarda 15 ponteiros
 - 12 primeiros apontam para blocos diretos (que contém dados)
 - 3 últimos para blocos indiretos
 - Primeiro aponta para bloco de índice de 1 nível, segundo para bloco de índice de dois níveis e terceiro para bloco de índice de 3 níveis

Métodos de alocação

- Alocação indexada (cont.):
 - Inode do UNIX:



Métodos de alocação

- Desempenho dos métodos de alocação: depende como será o uso do sistema
 - Alocação contígua: é adequada para qualquer tipo de acesso (apenas um acesso para alcançar bloco do disco)
 - Alocação encadeada: adequada para acesso sequencial, mas não para acesso direto
 - Sistemas operacionais podem suportar vários tipos
 - Arquivo criado para acesso sequencial: será encadeado e não possibilitará acesso direto
 - Arquivo criado para acesso direto: contíguo, mas usuário deve informar tamanho máximo
 - Alocação indexada: se bloco de índice estiver em memória, possibilita acesso direto
 - Caso contrário, pode precisar de muitos acessos

Gerenciamento de espaço livre

- Lista de espaços livres: estrutura de dados que registra blocos de disco livres
 - Criação de arquivo: pesquisa pelo espaço necessário
 - Exclusão de arquivo: espaço é adicionado à lista de blocos livres
 - Implementações da lista de espaços livres: nem sempre é lista propriamente dita
 - Vetor/mapa de bits
 - Lista encadeada
 - Agrupamento
 - Contagem
 - Mapas de espaços

Gerenciamento de espaço livre

- Vetor/mapa de bits:
 - Cada bloco é representado por um bit
 - Bit 1: bloco livre
 - Bit 0: bloco alocado
 - Ex.: Blocos livres 2, 3, 4, 5, 8, 9, 10
 - Mapa de bits: 00111100111
 - Busca por bloco livre:
 - Busca por palavra com valor diferente de zero (valor 0 indica que todos os blocos estão alocados)
 - Essa palavra é varrida em busca do primeiro bit 1 (primeiro bloco livre)
 - Cálculo do número do bloco:

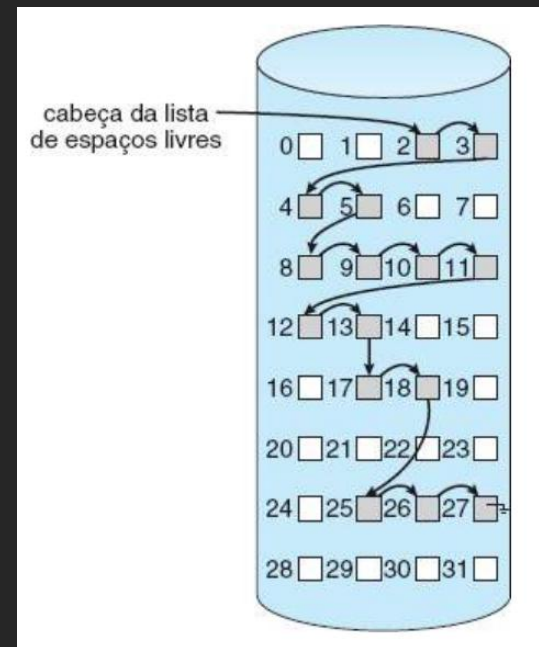
$(\text{número de bits por palavra}) \times (\text{número de palavras de valor 0}) + \text{deslocamento do primeiro bit 1}$

Gerenciamento de espaço livre

- Vetor de bits (cont.)
 - Vantagem: simplicidade de implementar, usando instruções de manipulação de bits
 - Desvantagem: estrutura ineficiente se o mapa de bits não estiver em memória
 - Disco de 1.3GB com blocos de 512 bytes: 332KB de mapa de bits
 - Disco de 1TB com blocos de 4KB: 32MB de mapa de bits

Gerenciamento de espaço livre

- Lista encadeada: blocos livres no disco são encadeados
 - Em memória é mantido ponteiro para lista
 - Ex.: blocos livres 2, 3, 4, 5, 8, 9, 10, 11, 12, 13, 17, 18, 25, 26 e 27
 - Ponteiro em memória para bloco 2



Gerenciamento de espaço livre

- Lista encadeada (cont):
 - Dificuldade: percorrer a lista é ineficiente, pois cada bloco precisa ser lido
 - Em geral não é necessário percorrer, porque usualmente os blocos são alocados de 1 em 1

Gerenciamento de espaço livre

- Agrupamento: lista encadeada onde cada bloco livre no disco armazena ponteiros para n blocos livres
 - $n-1$ primeiros ponteiros: indicam outros blocos livres
 - último ponteiro: aponta para próximo bloco livre que também armazena n ponteiros...
 - Vantagem: muitos blocos livres podem ser encontrados rapidamente (menos leituras de blocos são necessárias)

Gerenciamento de espaço livre

- Contagem: considera o fato de que geralmente vários blocos contíguos podem ser liberados, especialmente se houver alocação contígua ou de clusters
 - Cada nó na lista guarda número do bloco inicial e quantidade de blocos livres contíguos
 - Lista tende a ser mais curta
- Ao invés de usar lista encadeada também pode usar árvore balanceada para busca, inserção e remoção mais eficientes

Eficiência e desempenho

- Eficiência: uso eficiente de espaço depende diretamente dos algoritmos de alocação e de diretório
- Ex. 1: Unix pré-aloca inodes espalhados pelo volume
 - Busca-se alocar blocos de dados dos arquivos próximo aos blocos de inodes, reduzindo tempo de busca
- Ex. 2: BSD varia tamanho de cluster de alocação à medida que arquivo cresce
 - Clusters pequenos usados para arquivos pequenos e para último cluster de arquivo
 - Essa estratégia combina bom desempenho (menor tempo de busca) com pouca fragmentação interna

Eficiência e desempenho

- Eficiência (cont.):
 - Ex. 3: Alguns sistemas de arquivo mantêm campo “último tempo de acesso” na entrada de diretório/inode
 - Precisa atualizar diretório, mesmo quando arquivo é lido: ineficiente para arquivos acessados com muita frequência
 - Ex. 4: Tamanho de ponteiros para acesso a dados
 - Ponteiros de 32 bits (pequeno): gasta menos espaço em disco, mas tamanho do arquivo fica limitado a 2^{32} bytes = 4GB
 - Ponteiros de 128 bits (grande): arquivo “sem limite” de tamanho (2^{128} bytes) , mas gasta muito espaço em disco
 - Ex. 5: Solaris iniciais tinha estruturas de dados de tamanho fixo (tabela de processos e tabela de arquivos abertos)
 - Eficiente, mas impedia novo processo ser criado ou arquivo ser aberto: recompilar kernel era solução para aumentar tamanho

Eficiência e desempenho

- Desempenho: melhorado com uso de buffers/caches
 - Ex. 1: Memória de controladores de disco podem ser usadas como cache para guardar trilhas inteiras
 - Leituras de setores da mesma trilha podem ser feitas apenas transferindo dados do buffer do controlador para memória
 - Ex. 2: Alguns SOs reservam parte da memória principal para fazer cache de blocos de arquivos
 - Outros SOs (Solaris, Linux, Windows) armazenam dados de arquivos em cache através de caches de páginas em memória
 - Memória virtual unificada: lida tanto com páginas de processos quanto dados arquivos

Eficiência e desempenho

- Desempenho (cont.):
 - Ex. 3: Alguns UNIX e Linux usam cache de buffer unificado
 - Evita que acesso a arquivos com mapeamento em memória (mmap) leve a um cache duplo (cache do mmap + cache do sistema)
 - Cache duplo: desperdício de memória, desperdício de ciclos de CPU e possibilidade de mais inconsistências
 - Ex. 4: Versões do Solaris (< 2.5.1) não faziam distinção entre alocação de páginas para processo ou cache
 - Quando havia muita E/S, algoritmos de substituição de páginas expulsava páginas de processos
 - Solaris 2.6 e 7: dá prioridade maior para páginas de processos
 - Solaris 8: adicionou limite fixo para páginas de processos e cache
 - Solaris 9 e 10: novos algoritmos para minimizar atividade improdutiva

Eficiência e desempenho

- Desempenho (cont.):
 - Ex. 5: Sistemas costumam permitir gravações síncronas e assíncronas (opção da chamada `open()`)
 - Síncronas: na ordem em que são solicitadas e sem bufferização
 - Lentas mas úteis para transações atômicas de BD, por exemplo
 - Assíncronas: ocorre em cache e chamada retorna rapidamente
 - Ex. 6: Substituição de páginas de cache pelo LRU nem sempre é útil (leitura/gravação sequencial)
 - Técnicas free-behind e read-ahead podem auxiliar
 - Free-behind: remove página do buffer assim que próxima é solicitada
 - Read-ahead: página solicitada e várias subsequentes são armazenadas em cache
 - Ex. 7: Driver de disco grava páginas do cache de páginas em ordem para minimizar tempo de acesso

Recuperação

- Arquivos são mantidos em memória e disco: falha de sistema pode gerar inconsistências
- Verificação de consistência: usada para detectar problemas para posterior correção
- Varredura de todos metadados: muito demorado (minutos/horas)
 - Alternativa: bit status indica se metadado é estável ou não
 - Início da alteração de metadado: bit é ligado (instável)
 - Após alteração completa de metadado: bit é desligado (estável)
- fsck do UNIX: compara estruturas de diretório com blocos de dados em disco e tenta corrigir inconsistências
 - Ex. 1: Arquivos com alocação encadeada podem ser recuperados e estrutura de diretório recriada
 - Ex. 2: Perda entrada de diretório com alocação indexada: desastrosa

Recuperação

- fsck do UNIX: compara estruturas de diretório com blocos de dados em disco e tenta corrigir inconsistências
 - Ex. 1: Arquivos com alocação encadeada podem ser recuperados e estrutura de diretório recriada
 - Ex. 2: Perda entrada de diretório com alocação indexada: não há como encontrar blocos do arquivo
- Para permitir melhor verificação/recuperação, UNIX armazena em cache entradas de diretório apenas para leituras
 - Qualquer alteração que resulte em alocação de espaço ou alterações de metadados é feita sincronamente

Recuperação

- Sistemas de arquivos estruturados em log:
 - Todas alterações de metadados são armazenadas sequencialmente em log
 - Após gravação em log são consideradas **confirmadas** em processo continua
 - Entradas de log são reexecutadas
 - Conforme atualizações são feitas, ponteiro avança para indicar as **concluídas** e as incompletas
 - **Concluídas** são removidas do arquivo de log, que é um buffer circular
 - Se sistema cair:
 - Transações contidas nele não foram **concluídas** e são executadas
 - Problema: transação abortada (não **confirmada** antes da queda)
 - Alteração realizada por transação deve ser desfeita

Recuperação

- Outros sistemas de arquivos (WAFL e ZFS) não sobrepõem blocos com novos dados
- Quando transação é concluída, ponteiros dos metadados são atualizados para blocos novos: só então blocos antigos são disponibilizados
- ZFS acrescenta ainda uma soma de verificação de todos os blocos de dados e metadados, eliminando verificação de consistência

Recuperação

- Backup e restauração: indispensável
 - Backup: cópia em outro dispositivo que pode ser recuperada através da restauração
 - Backup incremental: minimiza espaço gasto
 - Dia 1: Copiar todos arquivos do disco (backup completo)
 - Dia 2: Copiar arquivos alterados desde Dia 1 (backup incremental)
 - Dia 3: Copiar arquivos alterados desde Dia 2
 - ...
 - Dia n: Copiar arquivos alterados desde Dia n-1
 - Ciclo se repete
 - Backup incremental permite restaurar disco inteiro iniciando a partir do completo e seguindo incrementais
 - Também permite restaurar arquivo a partir do backup anterior